

Strategy Design Pattern

Encapsulating Interchangeable Algorithms in OOP

CS202 — Object-Oriented Programming Seminar

Presenter: CS202 Seminar Group

Date: July 2026

Real-World Scenario: Navigation App

Imagine building a GPS Navigation App (Google Maps):

- **Driving:** Prioritizes highways, live traffic bypass & toll roads.
- **Walking:** Prioritizes sidewalks, crosswalks & pedestrian short-cuts.
- **Public Transit:** Syncs bus schedules, metro lines & transfer routes.
- **Bicycling:** Prioritizes dedicated bike paths & flat terrain.

Core Challenge: How can we support dynamic algorithm switching without cluttering client code with conditional `if-else` blocks?

Naive Implementation (Without Pattern)

Direct monolithic logic with `enum` and `if-else` blocks:

```
int main() {  
    Navigator navigator(TransportMode::DRIVING);  
    navigator.buildRoute("Home", "Downtown Office");  
  
    // Must switch mode via enum and run monolithic code block  
    navigator.setMode(TransportMode::WALKING);  
    navigator.buildRoute("Downtown Office", "City Park");  
  
    navigator.setMode(TransportMode::PUBLIC_TRANSIT);  
    navigator.buildRoute("City Park", "Home");  
  
    return 0;  
}
```

Problems with the Naive Approach

1. **Violates Open/Closed Principle (OCP):** Adding a new transport mode (e.g. `BICYCLING`) forces editing and recompiling `Navigator.h`.
2. **Violates Single Responsibility Principle (SRP):** `Navigator` manages context *and* every single routing algorithm.
3. **Monolithic Code Bloat:** Hundreds of lines of unrelated algorithm math inside one class.
4. **Testing Nightmare:** Cannot unit-test individual algorithms independently.

The Strategy Design Pattern

“ "Define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from clients that use it."

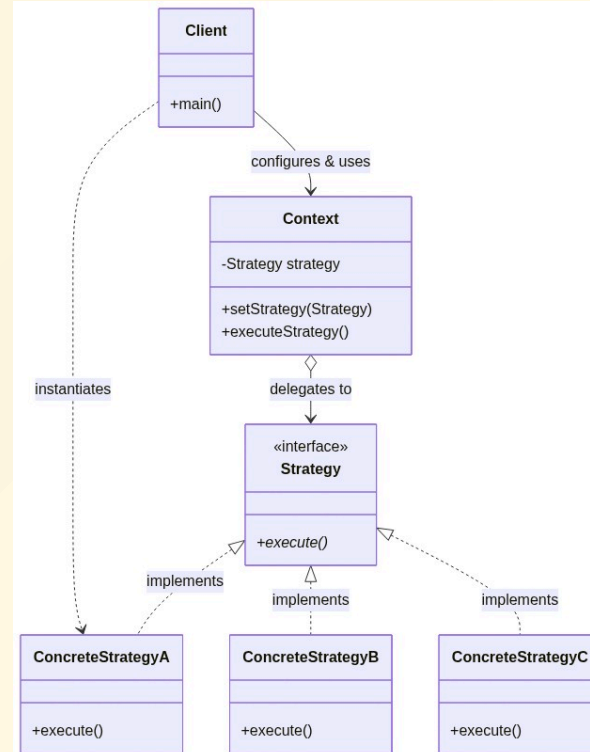
— *Gang of Four (GoF)*

”

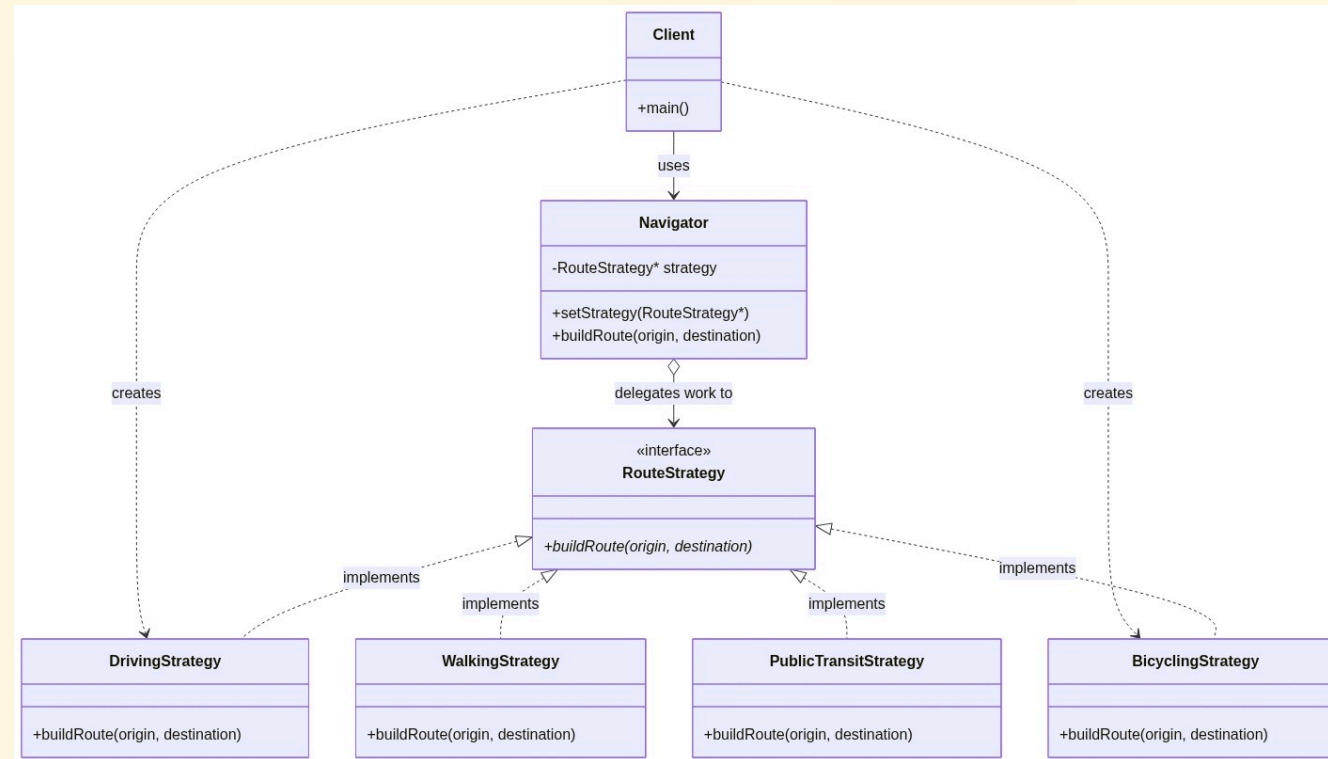
- **Category:** Behavioral Design Pattern.
- **Goal:** Isolate algorithms into dedicated strategy classes implementing a common interface.
- **Composition over Inheritance:** Context delegates algorithm execution to a strategy object reference.

General Class Diagram

The Client configures the Context with a Strategy object. The Context delegates work to the Strategy interface.



Navigation App Class Diagram



C++ Implementation: Strategy Classes

```
// RouteStrategy.h (Interface)
class RouteStrategy {
public:
    virtual ~RouteStrategy() = default;
    virtual void buildRoute(const std::string& origin, const std::string& destination) const = 0;
};

// DrivingStrategy.h (Concrete Strategy)
class DrivingStrategy : public RouteStrategy {
public:
    void buildRoute(const std::string& origin, const std::string& destination) const override {
        std::cout << " -> Mode: DRIVING (Highway priority & live traffic bypass)\n";
    }
};

// BicyclingStrategy.h (Extension without modifying existing code!)
class BicyclingStrategy : public RouteStrategy {
public:
    void buildRoute(const std::string& origin, const std::string& destination) const override {
        std::cout << " -> Mode: BICYCLING (Bike lanes & flat elevation profile)\n";
    }
};
```


C++ Implementation: Context & Clean Client

```
// main.cpp
int main() {
    DrivingStrategy driving;
    WalkingStrategy walking;
    BicyclingStrategy bicycling;

    Navigator navigator(&driving); // Context
    navigator.buildRoute("Home", "Office");

    // Dynamic runtime strategy swap
    navigator.setStrategy(&walking);
    navigator.buildRoute("Office", "Park");

    // Seamless OCP extension
    navigator.setStrategy(&bicycling);
    navigator.buildRoute("Home", "River Trail");
    return 0;
}
```

Pros and Cons of the Strategy Pattern

Pros	Cons
Open/Closed Principle: Add new algorithms without touching existing code.	Increased Object Count: Introduces additional class files to the codebase.
Single Responsibility: Isolates algorithm mechanics from context/UI logic.	Client Awareness: Clients must understand how strategies differ to pick the right one.
Runtime Swapping: Change behaviors on-the-fly via setter injection.	Indirection Overhead: Small virtual function call indirection.

Real-World Applications of Strategy

- **Web Development (Authentication & Payments):**

Passport.js (`LocalStrategy`, `OAuth2Strategy`) and Payment Gateways (`PayPal`, `CreditCard`, `ApplePay`).

- **Mobile Development (Layout Managers):**

Android `RecyclerView` (`LinearLayoutManager`, `GridLayoutManager`, `StaggeredGridLayoutManager`).

- **Game Development (AI Behaviors):**

NPC behavior state machines (`PatrolStrategy`, `AttackStrategy`, `FleeStrategy`).

- **Database Access & Sorting:**

C++ `std::sort` introsort switching between QuickSort, HeapSort, and InsertionSort.

Quiz — Question 1

What type of design pattern is the Strategy pattern?

a) Creational

b) Structural

c) Behavioral

d) Architectural

Quiz — Question 2

What is the primary intent of the Strategy pattern?

a) Adapt incompatible interfaces

b) Encapsulate interchangeable algorithms

c) Ensure single class instance

d) Simplify subsystem interface

Quiz — Question 3

Which SOLID principle is promoted when adding a new Strategy subclass?

a) Liskov Substitution Principle

b) Open/Closed Principle

c) Interface Segregation Principle

d) Dependency Inversion Principle

Quiz — Question 4

In the Strategy pattern, what is the role of the Context?

a) Instantiate strategies automatically

b) Hold a strategy reference & delegate execution

c) Convert parameters into JSON

d) Prevent client from picking strategies

Quiz — Question 5

Which of the following is a disadvantage of the Strategy pattern?

a) Couples context to concrete classes

b) Clients must know how strategies differ

c) Prevents runtime changes

d) Forces context rewrite on additions

Quiz — Question 6

How does Strategy differ from Template Method?

a) Strategy uses composition; Template uses inheritance

b) Strategy uses inheritance; Template uses composition

c) Strategy is structural; Template is creational

d) There is no difference between them

Quiz — Question 7

Why pass concrete strategies by pointer/reference in C++?

a) Forces compiler to inline code

b) Prevents object slicing & supports polymorphism

c) Creates a Singleton automatically

d) Blocks direct client invocation

Reviewer Roles & Feedback Loop

The Strategy design pattern seminar features peer reviewers responsible for:

- **Listening & Clarifying:** Ensuring code readability and design clarity.
- **Challenging Decisions:** Questioning strategy encapsulation boundaries and dynamic switching.
- **Reviewer Focus Areas:**
 - Does the real-world problem genuinely benefit from dynamic strategy swapping?
 - Do the class diagrams accurately illustrate composition (`Context o--> Strategy`)?
 - Are pros/cons/trade-offs presented objectively?

Conclusion & References

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Freeman, E., & Robson, E. (2004). *Head First Design Patterns*. O'Reilly Media.
- Refactoring Guru. (n.d.). *Strategy Design Pattern*. <https://refactoring.guru/design-patterns/strategy>
- SourceMaking. (n.d.). *Strategy Design Pattern*. https://sourcemaking.com/design_patterns/strategy

Questions & Answers

Thank you!